ASH

The Ashley Project
ASH Programming Language

Programming Guide and Reference Manual

Language, compiler (ashc), ASHIR, shasm assembler and the nlib library

October 2026
ASH-0001-EN

Copyright © 2026 Mallionic Inc. All rights reserved.
Document Number: ASH-0001-EN

This page intentionally left blank.

Copyright © 2026 Mallionic Inc. All rights reserved.

Notices and Disclaimers

This document is part of **The Ashley Project**. It describes the ASH programming language, the `ashc` compiler (version 1.0.0), the ASHIR intermediate representation, the `shasm` assembler (version 1.2.0) and the `nlib` system library that ships embedded in the compiler.

The Ashley Project is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 3, or (at your option) any later version. The Ashley Project is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

Every program listing in this manual that is presented as an *example* was compiled with `ashc` 1.0.0 while this document was being produced. Program output, exit statuses, ASHIR listings and disassembly listings shown next to an example are the real results of that build; they were not typed by hand. Results may differ with other compiler versions, other C libraries, other linkers or other operating systems.

The information in this manual is subject to change without notice. Code generation details (register choices, instruction order, stack layout, helper routine names) are described to help you understand and debug your programs; they are not a stable interface unless a section says so explicitly.

Software and workloads used in examples may have been optimized for the x86-64 Linux environment in which this document was built. Where a feature depends on the target (16-bit flat binaries, 32-bit ELF objects, Windows COFF objects, programs without the C library) the dependency is stated in the text.

x86, x86-64, Intel 64 and IA-32 are names used to identify processor architectures; Linux is a registered trademark of Linus Torvalds; Windows is a trademark of Microsoft Corporation; Java and GraalVM are trademarks of Oracle and/or its affiliates; other names and brands may be claimed as the property of others.

Copyright © 2026 Mallionic Inc. All rights reserved.

Revision History

Revision	Date	Description
1.0	October 2026	Initial release. Covers ashc 1.0.0, shasm 1.2.0 and the nlib libraries args, delay, math, null, stdbool, stdio, stdlib, string, syscalls and types.

Contents

1 Bare-Metal Programming: The VGA Text Screen 5

1.1 The Memory of a PC 5

1.2 How the Text Screen Works 5

1.3 The Skeleton of a Kernel 6

1.4 Hello, Screen 8

1.5 All the Colors 9

1.6 A Small VGA Driver 11

1.7 Going Further 16

Index 17

Copyright © 2026 Mallionic Inc. All rights reserved.

CHAPTER 1

BARE-METAL PROGRAMMING: THE VGA TEXT SCREEN

A program that runs *on bare metal* runs directly on the processor, with no operating system underneath: it is the first code of a kernel, a boot loader, a firmware or an embedded system. There is no `printf`, no files and no C library. To show something on the screen the program writes directly into the memory of the video card. This chapter explains how a PC screen works in text mode, builds a minimal kernel in ASH that writes to the screen at address `0xB8000`, and grows it into a small VGA driver. Every screen shown here is a real screenshot of the program running in the QEMU emulator.

1.1 THE MEMORY OF A PC

When a PC starts, the first megabyte of the address space has a fixed layout inherited from the original IBM PC. Some of these regions are not ordinary memory but windows into hardware:

Table 1-1. The first megabyte of a PC

Address	Size	Content
`0x00000`	1 KiB	Interrupt vector table (real mode).
`0x00400`	256 bytes	BIOS data area (keyboard buffer, timer ticks, screen size...).
`0x07C00`	512 bytes	Where the BIOS loads a boot sector.
`0xA0000`	64 KiB	VGA graphics memory (mode 13h: 320 x 200 pixels, one byte each).
`0xB8000`	32 KiB	VGA **text** memory: the characters you see on the screen.
`0xF0000`	64 KiB	The BIOS itself.
`0x100000`	...	Extended memory, above 1 MiB: where kernels are usually loaded.

1.2 HOW THE TEXT SCREEN WORKS

In text mode the screen has 80 columns and 25 rows. Each position, a *cell*, is stored as **two bytes** at address `0xB8000 + (row * 80 + column) * 2`:

Table 1-2. One cell of the text screen

Byte	Meaning
first	The character code, in code page 437: the ASCII characters plus line-drawing and block characters such as `0xDB` (a full block) and `0xC9` (a double-line corner).
second	The *attribute*: the foreground color in bits 0–3 and the background color in bits 4–6. Bit 7 makes the character blink (or selects a bright background).

Copyright © 2026 Mallionic Inc. All rights reserved.

Writing two bytes at the right address makes a character appear immediately: the video card reads this memory about seventy times per second.

Table 1-3. The 16 text colors

Value	Color	Value	Color	Value	Color
0	black	6	brown	12	light red
1	blue	7	light gray	13	light magenta
2	green	8	dark gray	14	yellow
3	cyan	9	light blue	15	white
4	red	10	light green	|	
5	magenta	11	light cyan	|	

An attribute is computed as `background * 16 + foreground`: `0x1F` is white (15) on blue (1), `0x4E` is yellow on red, `0x07` is the usual light gray on black.

1.3 THE SKELETON OF A KERNEL

The kernels of this chapter are started by QEMU's built-in *multiboot* loader (`qemu-system-i386 -kernel kernel.elf`), which loads them at 1 MiB in 32-bit protected mode. Two small files are shared by all of them; ch-multiboot explains them line by line.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 1-1. boot.asm: the entry point, written for shasm

BOOT.ASM

```
; boot.asm - multiboot header, stack and entry point of an ASH kernel
bits 32

MB_MAGIC equ 0x1BADB002
MB_FLAGS equ 0                    ; no special requests to the boot loader

section .multiboot
align 4
    dd MB_MAGIC
    dd MB_FLAGS
    dd -(MB_MAGIC + MB_FLAGS)     ; checksum: the three fields add up to 0

section .bss
align 16
stack_bottom:
    resb 16384                    ; 16 KiB of stack
stack_top:

section .text
global _start
extern kmain
_start:
    mov esp, stack_top            ; the loader does not give us a stack
    push ebx                      ; second argument: multiboot information
    push eax                      ; first argument: magic number 0x2BADB002
    call kmain                    ; kmain(magic, info), written in ASH
.hang:
    cli
    hlt
    jmp .hang
```

Example 1-2. linker.ld: where each part of the kernel goes

LINKER.LD

```
ENTRY(_start)
SECTIONS
{
    . = 1M;                                 /* the kernel is loaded at 1 MiB */
    .text   : { *(.multiboot) *(.text) *(.ashtext) }
    .rodata : { *(.rodata) }
    .data   : { *(.data) }
    .bss    : { *(.bss) }
}
```

The ASH part of every kernel is a function `kmain(magic, info)` marked `@Entry`, compiled with `@bits(32)` and `--no-libc`. The build always has the same steps:

SHELL

```
shasm --lang en -f elf32 -o boot.o boot.asm      # assemble the entry point
ashc kernel.ash --elf --no-libc -s -o kernel.o    # compile the ASH code
ld -m elf_i386 -T linker.ld -o kernel.elf boot.o kernel.o
qemu-system-i386 -kernel kernel.elf               # boot it
```

Copyright © 2026 Mallionic Inc. All rights reserved.

1.4 HELLO, SCREEN

The smallest useful kernel writes one line into video memory. It does not clear the screen, so the messages of the BIOS remain visible above it.

Example 1-3. hello.ash: writing characters into video memory

HELLO.ASH

```
@bits(32)

@Entry
pub func kmain(uint32 magic, uint32 info): void {
    uint8* video = 0xB8000;                  // the first cell of the screen
    string message = " Hello from an ASH kernel! ";
    int32 row = 9;                           // below the BIOS messages
    int32 i = 0;
    while (message[i] != 0) {
        int32 cell = (row * 80 + i) * 2;
        video[cell] = message[i];            // the character
        video[cell + 1] = 0x1F;              // white on blue
        i++;
    }
}
```

Table 1-4. Line by line: hello.ash

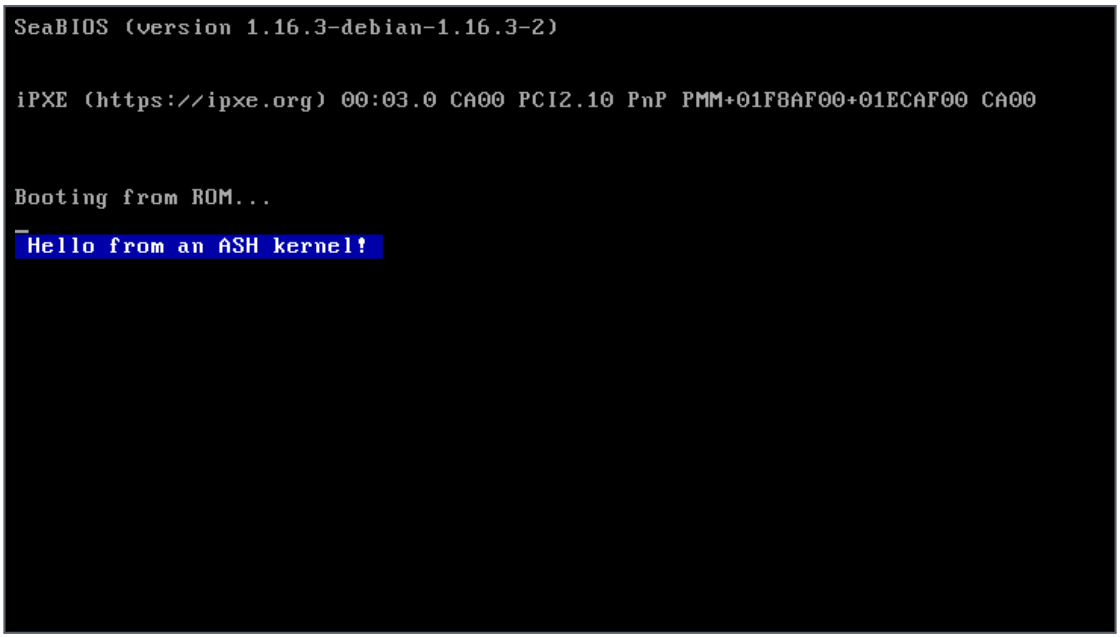
Code	What it does
`@bits(32)`	The kernel runs in 32-bit protected mode.
`pub func kmain(uint32 magic, uint32 info)`	Called by `boot.asm` with the two values that the loader leaves in `eax` and `ebx`.
`uint8* video = 0xB8000;`	A pointer to the first byte of text memory.
`int32 cell = (row * 80 + i) * 2;`	The byte offset of column `i` of row `row`.
`video[cell] = message[i];`	Stores the character code.
`video[cell + 1] = 0x1F;`	Stores its attribute: white (15) on blue (1).
(end of `kmain`)	Returns to `boot.asm`, which halts the processor with `cli` and `hlt`.

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc hello.ash --elf --no-libc -s -o hello.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o hello.elf boot.o hell
↪ o.o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 1-1. hello.ash running in QEMU



1.5 ALL THE COLORS

The next kernel shows every color the text screen offers. Below the BIOS messages ("Booting from ROM...") it draws sixteen squares of space characters, each with a different background, and a row of block characters in each foreground color.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 1-4. colors.ash: squares in the sixteen VGA colors

COLORS.ASH

```
@bits(32)

define VGA 0xB8000;

func put_at(int32 col, int32 row, int32 ch, int32 attr): void {
    uint8* cell = VGA + (row * 80 + col) * 2;
    cell[0] = ch;
    cell[1] = attr;
}

func text_at(int32 col, int32 row, string s, int32 attr): void {
    int32 i = 0;
    while (s[i] != 0) {
        put_at(col + i, row, s[i], attr);
        i++;
    }
}

// a block of 8 x 3 cells whose background is the given color
func square(int32 col, int32 row, int32 color): void {
    for (int32 y = 0; y < 3; y++) {
        for (int32 x = 0; x < 8; x++) {
            put_at(col + x, row + y, ' ', color << 4);
        }
    }
}

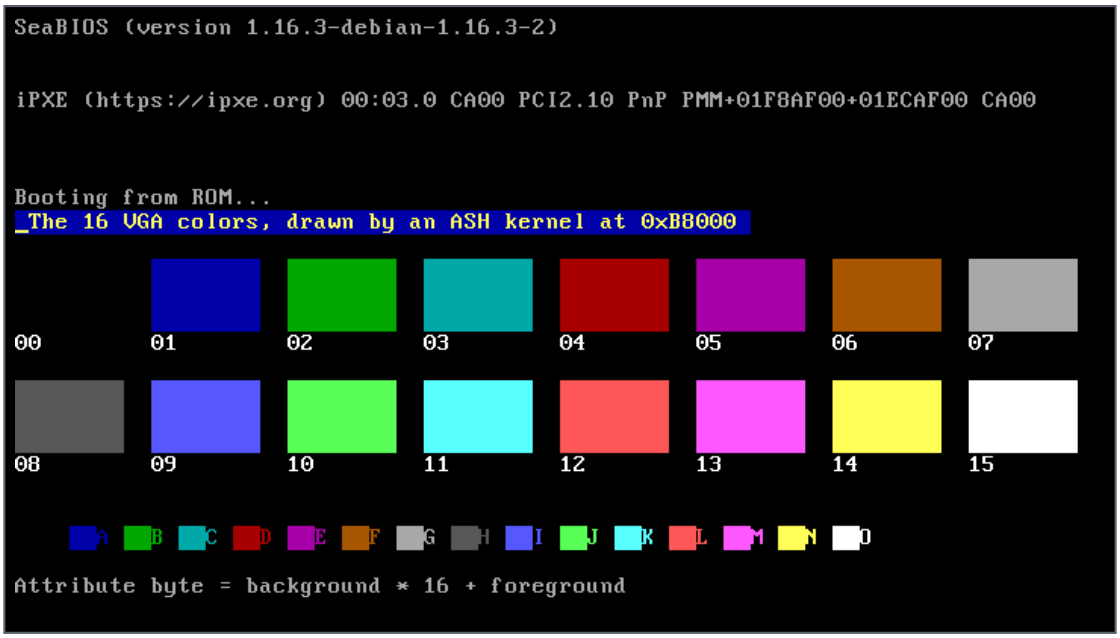
@Entry
pub func kmain(uint32 magic, uint32 info): void {
    text_at(0, 8, " The 16 VGA colors, drawn by an ASH kernel at 0xB8000 ", 0x1E);
    for (int32 c = 0; c < 16; c++) {
        int32 col = (c % 8) * 10;
        int32 row = 10 + (c / 8) * 5;
        square(col, row, c);
        put_at(col, row + 3, '0' + c / 10, 0x0F);
        put_at(col + 1, row + 3, '0' + c % 10, 0x0F);
    }
    for (int32 c = 1; c < 16; c++) {
        put_at(c * 4, 21, 0xDB, c);              // a full block in color c
        put_at(c * 4 + 1, 21, 0xDB, c);
        put_at(c * 4 + 2, 21, 'A' + c - 1, c);   // a letter in color c
    }
    text_at(0, 23, "Attribute byte = background * 16 + foreground", 0x07);
}
```

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc colors.ash --elf --no-libc -s -o colors.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o colors.elf boot.o col
↪ ors.o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 1-2. colors.ash: the sixteen colors below the BIOS messages



Square 0 is black on black, so it is invisible; squares 8 to 15 are the bright colors, which QEMU's VGA shows as backgrounds because blinking is disabled.

1.6 A SMALL VGA DRIVER

Writing cells one by one quickly becomes tedious. The file `vga.ash` collects the operations every kernel needs: clearing the screen, writing text that wraps and scrolls, printing numbers in decimal and hexadecimal, drawing frames and bars, and moving the blinking hardware cursor. The cursor is not in video memory: it is a register of the video card's CRT controller, reached through the I/O ports `0x3D4` (register index) and `0x3D5` (value) with the `out` instruction.

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 1-5. vga.ash: a reusable text-mode driver

VGA.ASH

```
// vga.ash - a small driver for the VGA text screen (80 x 25 cells at 0xB8000)
define VGA_TEXT 0xB8000;
define VGA_COLS 80;
define VGA_ROWS 25;

general static int32 vga_row = 0
general static int32 vga_col = 0
general static int32 vga_color = 0x07

// attribute byte: background color in bits 4-6, foreground color in bits 0-3
func vga_attr(int32 fg, int32 bg): int32 {
    return (bg << 4) | fg;
}

func vga_put(int32 col, int32 row, int32 ch, int32 attr): void {
    uint8* cell = VGA_TEXT + (row * VGA_COLS + col) * 2;
    cell[0] = ch;
    cell[1] = attr;
}

func vga_clear(int32 attr): void {
    for (int32 i = 0; i < VGA_COLS * VGA_ROWS; i++) {
        vga_put(i % VGA_COLS, i / VGA_COLS, ' ', attr);
    }
    vga_row = 0;
    vga_col = 0;
}

func vga_goto(int32 col, int32 row): void {
    vga_col = col;
    vga_row = row;
}

// move the blinking hardware cursor: CRT controller registers 14 and 15
func vga_cursor(int32 col, int32 row): void {
    int32 pos = row * VGA_COLS + col;
    int32 high = (pos >> 8) & 255;
    int32 low = pos & 255;
    asm {
        mov dx, 0x3D4
        mov al, 14
        out dx, al
        mov dx, 0x3D5
        mov eax, high
        out dx, al
        mov dx, 0x3D4
        mov al, 15
        out dx, al
        mov dx, 0x3D5
        mov eax, low
        out dx, al
    }
}

// move every line up by one and clear the last line
func vga_scroll(): void {
    uint8* screen = VGA_TEXT;
    for (int32 i = 0; i < (VGA_ROWS - 1) * VGA_COLS * 2; i++) {
        screen[i] = screen[i + VGA_COLS * 2];
```

Copyright © 2026 Mallionic Inc. All rights reserved.

VGA.ASH (CONTINUED)

```
    }
    for (int32 c = 0; c < VGA_COLS; c++) {
        vga_put(c, VGA_ROWS - 1, ' ', vga_color);
    }
    vga_row = VGA_ROWS - 1;
}

func vga_putc(int32 ch): void {
    if (ch == '\n') {
        vga_col = 0;
        vga_row = vga_row + 1;
    } else {
        vga_put(vga_col, vga_row, ch, vga_color);
        vga_col = vga_col + 1;
        if (vga_col == VGA_COLS) {
            vga_col = 0;
            vga_row = vga_row + 1;
        }
    }
    if (vga_row == VGA_ROWS) {
        vga_scroll();
    }
}

func vga_puts(uint8* s): void {
    int32 i = 0;
    while (s[i] != 0) {
        vga_putc(s[i]);
        i++;
    }
}

func vga_dec(uint32 v): void {
    uint8[12] digits;
    int32 n = 0;
    if (v == 0) {
        vga_putc('0');
        return;
    }
    while (v > 0) {
        digits[n] = '0' + v % 10;
        v = v / 10;
        n++;
    }
    while (n > 0) {
        n--;
        vga_putc(digits[n]);
    }
}

func vga_hex(uint32 v): void {
    vga_puts("0x");
    for (int32 shift = 28; shift >= 0; shift = shift - 4) {
        int32 d = (v >> shift) & 15;
        if (d < 10) { vga_putc('0' + d); } else { vga_putc('A' + d - 10); }
    }
}

// a frame drawn with the double-line characters of code page 437
func vga_frame(int32 col, int32 row, int32 w, int32 h, int32 attr): void {
    vga_put(col, row, 0xC9, attr);
```

Copyright © 2026 Mallionic Inc. All rights reserved.

VGA.ASH (CONTINUED)

```
    vga_put(col + w - 1, row, 0xBB, attr);
    vga_put(col, row + h - 1, 0xC8, attr);
    vga_put(col + w - 1, row + h - 1, 0xBC, attr);
    for (int32 x = 1; x < w - 1; x++) {
        vga_put(col + x, row, 0xCD, attr);
        vga_put(col + x, row + h - 1, 0xCD, attr);
    }
    for (int32 y = 1; y < h - 1; y++) {
        vga_put(col, row + y, 0xBA, attr);
        vga_put(col + w - 1, row + y, 0xBA, attr);
    }
}

// a bar of 'full' cells out of 'size', in the given color
func vga_bar(int32 col, int32 row, int32 size, int32 full, int32 attr): void {
    for (int32 i = 0; i < size; i++) {
        if (i < full) {
            vga_put(col + i, row, 0xDB, attr);
        } else {
            vga_put(col + i, row, 0xB0, 0x08);
        }
    }
}
```

Table 1-5. Functions of vga.ash

Function	What it does
`vga_attr(fg, bg)`	Builds an attribute byte from two colors of Table 1-3.
`vga_put(col, row, ch, attr)`	Writes one character with one attribute at a position.
`vga_clear(attr)`	Fills the screen with spaces of the given attribute and moves the text position to the top left.
`vga_goto(col, row)`	Sets the position where the next text goes.
`vga_putc(ch)`	Writes one character at the text position in `vga_color`; handles `\n`, line wrapping and scrolling.
`vga_puts(s)`	Writes a string with `vga_putc`.
`vga_dec(v)`, `vga_hex(v)`	Write a number in decimal, or as `0x` followed by eight hexadecimal digits.
`vga_frame(col, row, w, h, attr)`	Draws a double-line frame with the characters `0xC9`, `0xBB`, `0xC8`, `0xBC`, `0xCD`, `0xBA`.
`vga_bar(col, row, size, full, attr)`	Draws a bar of `size` cells, the first `full` of them solid (`0xDB`), the rest shaded (`0xB0`).
`vga_cursor(col, row)`	Moves the hardware cursor: writes the high and low byte of `row * 80 + col` to CRT registers 14 and 15.
`vga_scroll()`	Moves every line up by one and clears the last line.

The following kernel uses the driver to draw a control panel:

Copyright © 2026 Mallionic Inc. All rights reserved.

Example 1-6. dash.ash: a control panel drawn with vga.ash

DASH.ASH

```
@bits(32)
import "vga.ash";

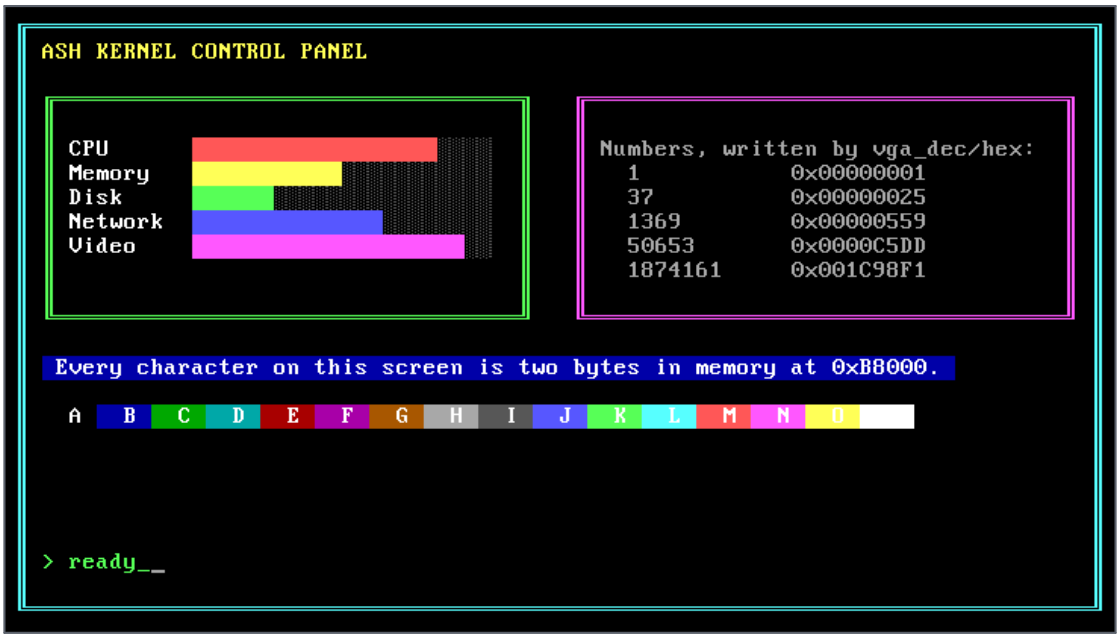
@Entry
pub func kmain(uint32 magic, uint32 info): void {
    vga_clear(vga_attr(7, 0));
    vga_frame(0, 0, 80, 25, vga_attr(11, 0));
    vga_goto(2, 1);
    vga_color = vga_attr(14, 0);
    vga_puts("ASH KERNEL CONTROL PANEL");
    vga_frame(2, 3, 36, 10, vga_attr(10, 0));
    vga_frame(41, 3, 37, 10, vga_attr(13, 0));
    string[] names = {"CPU", "Memory", "Disk", "Network", "Video"};
    int32[] levels = {18, 11, 6, 14, 20};
    int32[] colors = {12, 14, 10, 9, 13};
    for (int32 i = 0; i < 5; i++) {
        vga_goto(4, 5 + i);
        vga_color = vga_attr(15, 0);
        vga_puts(names[i]);
        vga_bar(13, 5 + i, 22, levels[i], vga_attr(colors[i], 0));
    }
    vga_goto(43, 5);
    vga_color = vga_attr(7, 0);
    vga_puts("Numbers, written by vga_dec/hex:");
    int32 v = 1;
    for (int32 r = 0; r < 5; r++) {
        vga_goto(45, 6 + r);
        vga_dec(v);
        vga_goto(57, 6 + r);
        vga_hex(v);
        v = v * 37;
    }
    vga_goto(2, 14);
    vga_color = vga_attr(15, 1);
    vga_puts(" Every character on this screen is two bytes in memory at 0xB8000. ");
    vga_goto(2, 16);
    for (int32 c = 0; c < 16; c++) {
        vga_color = vga_attr(c, 0);
        vga_putc(0xDB);
        vga_putc(0xDB);
        vga_color = vga_attr(15, c);
        vga_putc('A' + c);
        vga_putc(' ');
    }
    vga_goto(2, 22);
    vga_color = vga_attr(10, 0);
    vga_puts("> ready_");
    vga_cursor(10, 22);
}
```

SHELL SESSION

```
$ shasm --lang en -f elf32 -o boot.o boot.asm
$ ashc dash.ash --elf --no-libc -s -o dash.o
$ ld -m elf_i386 -z noexecstack --no-warn-rwx-segments -T linker.ld -o dash.elf boot.o dash.
↪ o
```

Copyright © 2026 Mallionic Inc. All rights reserved.

Figure 1-3. dash.ash: frames, bars, numbers and colors



1.7 GOING FURTHER

- ch-multiboot explains the multiboot header, the linker script and the information the boot loader passes to the kernel.
- ch-dos shows real mode, where the same text memory is reached as segment `0xB800`.
- A keyboard driver reads port `0x60` when interrupt 1 arrives; an interrupt handler is a `@naked` function that ends with `iret` (asm-naked).

Copyright © 2026 Mallionic Inc. All rights reserved.

INDEX

0

0xB8000 5

A

attribute byte 5

B

bare metal 5

T

text mode 5

V

VGA 5

Copyright © 2026 Mallionic Inc. All rights reserved.